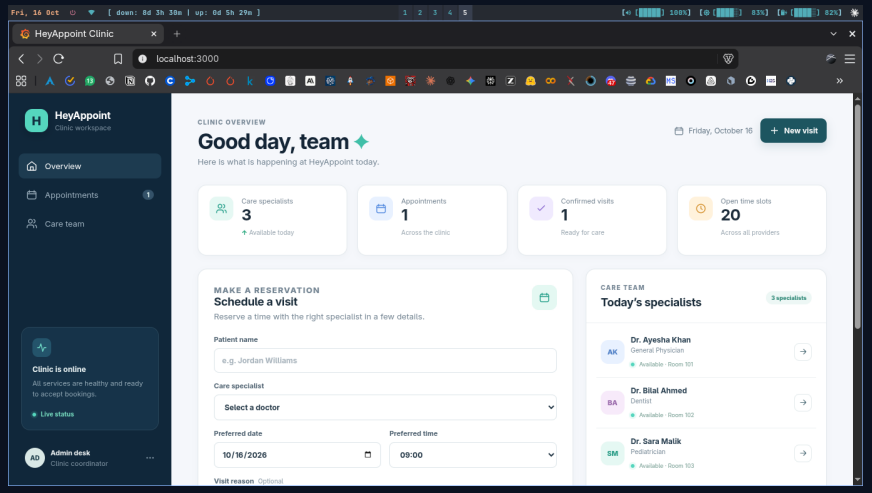
HeyAppoint Clinic

End-to-end clinic appointments with a verifiable DevOps delivery pipeline

Book. Operate. Observe. Ship safely.

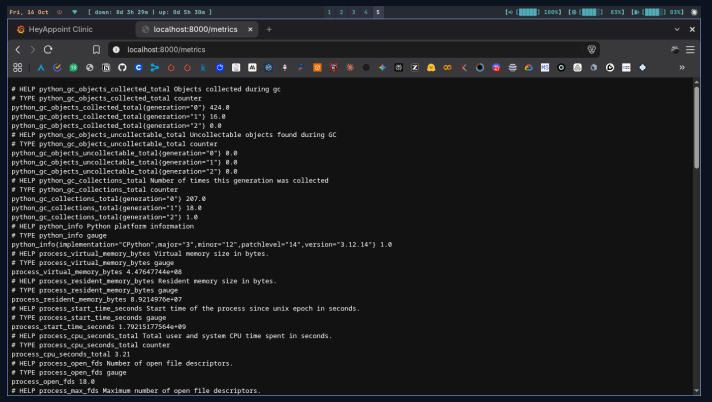
FastAPI + PostgreSQL + React + Docker + GitHub Actions + Kubernetes



The product

A usable clinic workflow, not a blank demo page

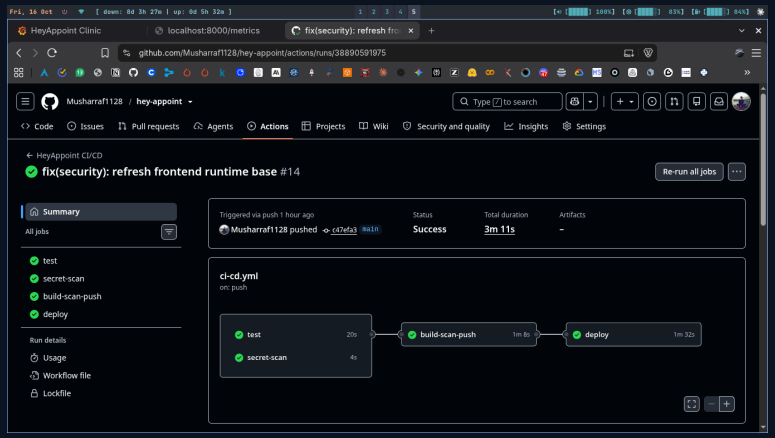
- Patients choose a doctor, date, time, and reason
- Staff confirm or cancel appointments
- Duplicate doctor/date/time bookings return HTTP 409
- REST API includes GET, POST, PUT, and DELETE endpoints



Quality and delivery gates

Every release is tested, built, scanned, and smoke-deployed

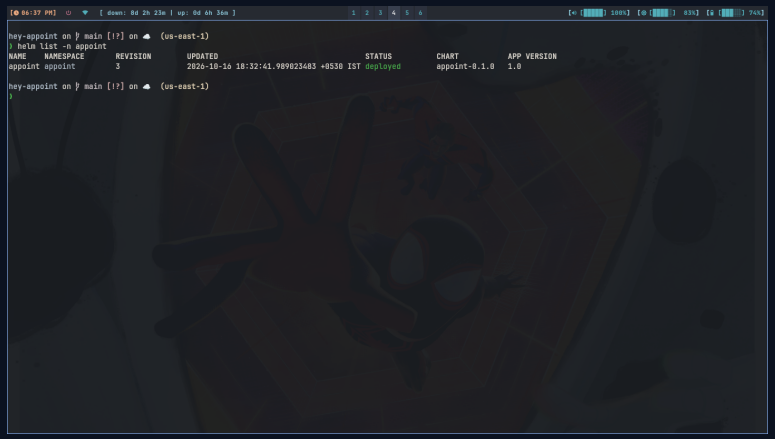
- Six API tests run before image builds
- Frontend production build runs in CI
- Backend and frontend images use commit SHA tags
- Trivy HIGH/CRITICAL findings fail the release gate
- GHCR publication follows successful checks



Containers and Kubernetes

Reproducible local and cluster deployment

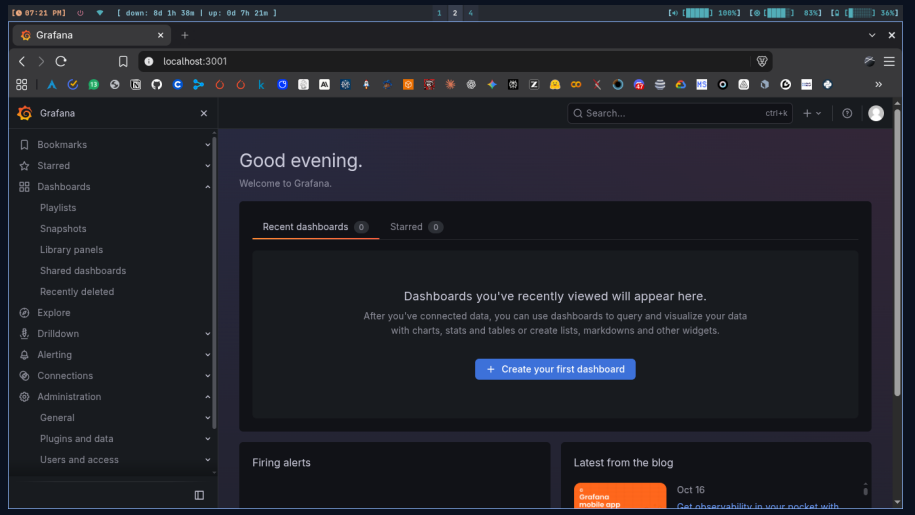
- Docker Compose runs frontend, backend, and PostgreSQL
- Multi-stage frontend image runs as non-root Nginx
- Helm deploys two application replicas and ClusterIP services
- Ingress routes / to frontend and /api to backend
- Health probes and HPA support operations



Observability

Prometheus-compatible metrics with a ready Grafana dashboard

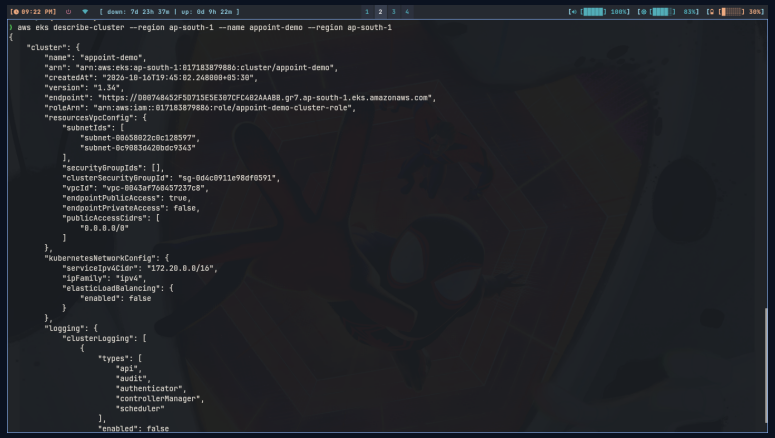
- Backend exposes /metrics
- ServiceMonitor discovers the API across namespaces
- Prometheus target is verified UP
- Grafana dashboard shows request rate and p95 latency



AWS infrastructure

Terraform provisions a disposable EKS demonstration environment

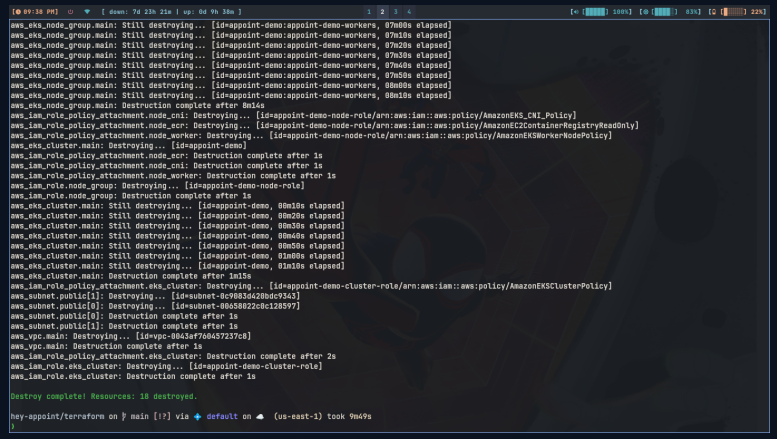
- VPC with two public subnets in separate Availability Zones
- Managed EKS control plane and worker node group
- Outputs provide cluster endpoint and kubeconfig command
- Evidence captured for VPC, subnets, cluster, and node group



Evidence and handoff

The project is documented for grading and safe teardown

- 27 numbered screenshots are organized in submission/
- submission/README.md displays the evidence gallery
- Terraform destroy completed after the AWS demonstration
- Repository README explains local run, CI/CD, Kubernetes, and monitoring



Key learnings and next steps

What this project demonstrates and how it can evolve

- Automated quality gates make releases repeatable and auditable
- Infrastructure, application delivery, and observability are documented together
- Non-root images, dependency updates, and Trivy scanning reduce delivery risk
- Next evolution: ArgoCD GitOps sync, managed secrets, and production ingress TLS